

Тестовое задание — CV/ML Engineer

Кратко

Вам выдаётся запись с многокамерного стенда (rosbag2) и метаданные к ней. Нужно построить и сравнить несколько методов оценки траектории движущейся камеры, а затем — 3D-реконструкцию сцены. Финал — короткая презентация результатов.

Задание оценивает не «правильный ответ», а **инженерный и исследовательский подход**: как вы готовите данные, ставите эксперименты, измеряете ошибку, объясняете её причины и обосновываете улучшения.

Ограничений на стек нет. Решение — на Python приветствуется, но не обязательно. **ROS устанавливать не нужно** (см. подсказки ниже).

Описание стенда

- **Большая камера** — статичная, смотрит на сцену со стороны (RGB).
- **Малая камера** — подвижная, RGB-D, закреплена на ручном стабилизаторе (гимбале), человек водит ею по сцене.
- На сцене и на стабилизаторе размещены **ArUco-маркеры** (словарь `DICT_4X4_50`):
 - **id20, id21, id23** — копланарная «доска» на столе (задаёт **систему координат сцены**). Размещена на столе **намеренно**: малая камера периодически её видит и может локализоваться самостоятельно.
 - **id1** — маркер на **голове** малой камеры (отслеживаемое тело).
 - **id0** — маркер на **ручке** стабилизатора.
- Измеренные размеры маркеров: `id20 = id21 = id23 = id1 = 90 мм`, `id0 = 42 мм`.
- Связка ручка↔голова (id0↔id1) в этой записи практически **жёсткая** (типичное разнесение ~345 мм).

Идея сетапа: большая статичная камера наблюдает сцену «снаружи» (outside-in), малая камера движется внутри сцены (inside-out). Доска на столе даёт малой камере независимый якорь для самолокализации.

Выдаваемые данные

```
data/raw/rosbag2_<ts>/      # запись rosbag2 (.db3 + metadata.yaml)
```

- Топики обеих камер (цвет; для малой — также глубина), их `metadata.yaml`, таблицы `topics/schema`.
- Метаданные: интринзики камер, размеры маркеров, общая шкала времени для обеих камер (в сообщениях есть и bag-таймстемпы, и истинные header-таймстемпы захвата).

⚠ **Подсказка по данным.** Запись делалась «в поле», и её целостность местами под вопросом — не вся запись читается стандартными средствами от начала до конца. Отнеситесь к чтению данных оборонительно: проверяйте, сколько сообщений реально удалось прочитать, и аккуратно обрабатывайте момент, где чтение перестаёт работать. Часть подхода к заданию — **обнаружить и корректно обойти** эту проблему, а не споткнуться об неё.

Часть 0. Подготовка данных

0.1 Извлечение и нарезка на чанки.

- Прочитайте записанные кадры обеих камер, сопоставьте их по времени (камеры делят одну шкалу часов).
- Разбейте пригодную часть записи на несколько **временных сегментов (чанков)** — логически осмысленных кусков движения. Обоснуйте, как выбирали границы.
- Зафиксируйте, какая часть записи реально пригодна к работе и почему.

0.2 Pseudo-Ground-Truth (эталон).

- Постройте максимально надёжную траекторию малой камеры в координатах сцены, используя **всю доступную информацию** (большая камера + маркеры + самолокализация малой камеры по доске + сглаживание).
- Эта траектория станет **эталонном (pseudo-GT)**, относительно которого вы будете оценивать все остальные методы в основной части.
- Оцените и обоснуйте **собственную точность** эталона (например, сравнив фьюзинг с независимыми наблюдениями, которые в него не вошли). Без оценки точности эталон использовать нельзя — это важный пункт.
- (доп.) предоставить мат. выкладку о связности систем координат

Артефакт части 0: траектория-эталон (например, в формате TUM: `timestamp tx ty tz qx qy qz qw`) + краткое описание метода и его точности.

Часть 1 (основная). Оценка траектории — эксперименты и анализ

Цель — получить траекторию малой камеры **несколькими независимыми способами** и честно их сравнить с эталоном.

1.1 Camera-only трекинг (inside-out).

- Восстановите траекторию малой камеры **только по её собственному видеопотоку**, без маркеров и без большой камеры (SfM / визуальная одометрия / SLAM / выученные методы — на ваш выбор, можно несколько).
- Траектория camera-only обычно с точностью до подобия (масштаб/гейдж) — приведите её к эталону корректным выравниванием (например, Sim3/Umeyama) и считайте ATE.

1.2 Outside-in только по маркеру камеры.

- Восстановите траекторию малой камеры по наблюдению **только маркера id1** большой (статичной) камерой. Это «чистый» outside-in по одному маркеру.

1.3 (доп.) Оптимизация фактор-графом по паре маркеров.

- Используйте **пару маркеров стабилизатора {id1 (голова) + id0 (ручка)}** как жёсткое тело и постройте оценку траектории через **оптимизацию фактор-графом** (на ваш выбор инструмент: GTSAM / Ceres / g2o / mrob / собственная реализация).
- Покажите, что это даёт по сравнению с одиночным маркером id1.

1.4 Сравнение и анализ (ключевой пункт части).

- Сведите все методы (1.1–1.3) к эталону и постройте сравнительную таблицу/графики (ATE: медиана / среднее / p95; при возможности — повороты отдельно).
- **Обоснуйте ошибку**: где и почему каждый метод ошибается (например, поведение одиночного маркера в быстром движении, наблюдаемость глубины планарного маркера, слабые по текстуре кадры для camera-only и т.д.). Подкрепляйте выводы данными, а не общими словами.
- **Выдвиньте гипотезу по улучшению** наименее точного метода и объясните, почему она должна сработать.
 - (доп.) Подкрепите гипотезу **ссылкой на литературу** (статья/метод).

Часть 2 (основная). 3D-реконструкция сцены

2.1 Реконструкция несколькими методами.

- Постройте 3D-реконструкцию сцены (или её фрагмента/чанка) минимум **двумя разными методами** — например, классический по RGB-D с известными позами, и/или фотограмметрия/SfM-MVS, и/или обучаемые feed-forward методы. Метод(ы) — на ваш выбор, важно разнообразие подходов.

2.2 Траектория как prior.

- Покажите, как использование **известной траектории камеры** (например, outside-in позы малой камеры) в качестве **prior** влияет на результат: на точность поз и/или на качество и метричность реконструкции.
- Сравните результат «без prior» vs «с prior».

2.3 Лучшее качество.

- Сведите воедино, какой метод и с какими настройками даёт **наилучшее качество** реконструкции, и обоснуйте критериями (покрытие, точность геометрии относительно опорной реконструкции/эталона, метричность масштаба, артефакты).

Артефакты части 2: реконструкции (mesh/облако), сравнительные метрики/визуализации, краткое описание методов и настроек.

Часть 3 (финал). Презентация

Подготовьте **краткую презентацию (~10 минут)** с результатами:

- что и как делали (коротко по этапам),
- сравнительные результаты трекинга и реконструкции (таблицы/графики/видео),
- главные выводы: какой метод и почему лучший, где основные источники ошибки,
- гипотезы по улучшению.

Презентация важнее «красоты кода»: мы хотим увидеть, как вы **думаете и аргументируете**.

Практические подсказки

- **ROS ставить не нужно.** `rosbag2 .db3` можно читать как SQLite + десериализовать CDR-сообщения библиотекой `rosbags` (Ternaris). На машине может не быть `sqlite3` CLI — используйте модуль `sqlite3` из Python.
- ArUco живёт в `opencv-contrib-python` (модуль `cv2.aruco`), не в обычном `opencv-python`.
- Для сопоставления кадров двух камер используйте **header-таймстемпы захвата** (не bag-таймстемпы): обе камеры делят одну шкалу времени, типичный сдвиг — единицы мс.
- Метрический масштаб берите из **измеренных размеров маркеров**.
- Система координат сцены: начало — центр `id20`, ось Z — нормаль доски в сторону большой камеры, X — направление `id20→id21`, $Y = Z \times X$.

Удачи! Вопросы по постановке — приветствуются